OBJECTIVE-C · ПРЕЗЕНТАЦИЯ 2 ИЗ 3

Objective-C: надёжность и структура

Обработка ошибок · управление сложностью кода · обобщённые типы · коллекции · ввод-вывод

План презентации

ЧАСТЬ 1

Основы

- Общие сведения: история, особенности, версия
- Как запустить: Docker, онлайн, локально
- Данные и переменные, владение
- Функции, рекурсия, замыкания (блоки)
- Условия, циклы, итераторы

ЧАСТЬ 2

Надёжность и структура

- Исключения, NSError, assert
- Префиксы, заголовки, модули, библиотеки
- Обобщённые типы
- Коллекции Foundation
- Ввод-вывод: консоль и файлы

ЧАСТЬ 3

Модель исполнения

- Конкурентность: NSThread, GCD, NSOperation
- ООП, протоколы, категории, ФП
- Специфические конструкции: сообщения, KVC/KVO
- Среда выполнения: рантайм, objc_msgSend
- Управление памятью: MRC, ARC

Две модели ошибок

	Исключения `NSException`	Ошибки `NSError`
Для чего	**логические ошибки в коде**: выход за границы, неверный аргумент, нарушение инварианта	**ожидаемые сбои**: нет файла, нет сети, неверный ввод пользователя
Механизм	`@throw` / `@try` / `@catch` / `@finally`	возвращаемое значение + выходной параметр `NSError **`
Что дальше	обычно — завершение программы	обработать и продолжить работу
Стоимость	дорого; при ARC не освобождает объекты на пути раскрутки*	обычный вызов функции
Пример в Foundation	`[array objectAtIndex:100]`	`[fm attributesOfItemAtPath:p error:&err]`

* По умолчанию ARC не exception-safe: при раскрутке стека retain/release пропускаются. Флаг `-fobjc-arc-exceptions` это исправляет ценой производительности.

@try / @catch / @finally

01_exceptions.m

```objc
void validateAge(NSInteger age) {
    if (age < 0) @throw [ValidationException exceptionWithField:@"age"];
    if (age > 150)
        [NSException raise:NSInvalidArgumentException format:@"age %ld?!", (long)age];
}
```

01_exceptions.m

```objc
NSArray *values = @[@10, @-5, @200];
for (NSNumber *v in values) {
    @try {
        validateAge(v.integerValue);
        NSLog(@"%@: ok", v);
    } @catch (ValidationException *e) {     // от частного к общему
        NSLog(@"%@: validation, field=%@", v, e.userInfo[@"field"]);
    } @catch (NSException *e) {
        NSLog(@"%@: %@ - %@", v, e.name, e.reason);
    } @finally {
        NSLog(@"%@: finally", v);           // выполняется всегда
    }
}
```

```
$ ./01_exceptions
10: ok
10: finally
-5: validation, field=age
-5: finally
200: NSInvalidArgumentException - age 200?!
200: finally
```

NSError: домен, код, userInfo

02_errors.m

```
static NSString *const ConfigErrorDomain = @"ConfigErrorDomain";
typedef NS_ENUM(NSInteger, ConfigError) {
    ConfigErrorMissing = 1,
    ConfigErrorInvalid = 2,
};
```

- **domain** — строка, указывающая источник ошибки: системные домены `NSPOSIXErrorDomain`, `NSCocoaErrorDomain` или домен приложения
- **code** — номер ошибки внутри домена, удобно задавать через `NS_ENUM`
- **userInfo** — словарь с подробностями: текст для пользователя — ключ `NSLocalizedDescriptionKey`, исходная ошибка, вызвавшая текущую, — ключ `NSUnderlyingErrorKey`
- Правило: успех определяется по **возвращаемому значению**, а не по `error`

Использование NSError

02_errors.m

```m
NSError *err = nil;
NSDictionary *cfg = loadConfig(@"/no/such/file.conf", &err);
if (!cfg) {
    NSLog(@"domain=%@ code=%ld", err.domain, (long)err.code);
    NSLog(@"message: %@", err.localizedDescription);
    NSError *cause = err.userInfo[NSUnderlyingErrorKey];
    NSLog(@"cause: %@ / %ld", cause.domain, (long)cause.code);
}

cfg = loadConfig(@"/etc/hostname", NULL);         // ошибка не интересна — NULL
NSLog(@"loaded: %d", cfg != nil);
```

```
$ ./02_errors
domain=ConfigErrorDomain code=1
message: no config at /no/such/file.conf
cause: NSPOSIXErrorDomain / 2
loaded: 1
```

Особенность GNUstep: `stringWithContentsOfFile:encoding:error:` при отсутствии файла возвращает nil, но **не заполняет** `NSError` (в Apple Foundation заполняет). `NSFileManager` заполняет везде.

Отладка: assert и NSAssert

03_assert.m

```objc
@interface BankAccount : NSObject
@property (nonatomic) double balance;
- (void)withdraw:(double)sum;
@end

@implementation BankAccount
- (void)withdraw:(double)sum {
    NSParameterAssert(sum > 0);                          // проверка аргумента
    NSAssert(sum <= self.balance, @"not enough money: %.2f > %.2f", sum, self.balance);
    self.balance -= sum;
}
@end
```

03_assert.m

```objc
double safeSqrtArg(double x) {
    NSCAssert(x >= 0, @"negative: %f", x);   // NSCAssert — для C-функций
    return x;
}
```

assert vs NSAssert

03_assert.m

```
int *ptr = NULL;
assert(ptr != NULL && "pointer must not be NULL");  // abort() + сообщение
```

Средство	Где	Что делает	Отключается
`assert(cond)`	C, `<assert.h>`	печатает условие, `abort()`	`-DNDEBUG`
`NSAssert` / `NSParameterAssert`	методы	бросает `NSInternalInconsistencyException`	`-DNS_BLOCK_ASSERTIONS`
`NSCAssert` / `NSCParameterAssert`	C-функции	то же, без `self`	`-DNS_BLOCK_ASSERTIONS`

> В релизной сборке проверки обычно отключены, поэтому код внутри assert не должен иметь побочных эффектов.

Пространства имён: их нет

- Все классы, функции, глобальные символы — в **одном** пространстве
- Вместо namespace — **префиксы**: `NS` (NeXTSTEP), `GS` (GNUstep), `UI` (UIKit), `CF`, …
- Apple рекомендует **3 буквы** для своего кода: `GMPoint`, `ACMUser`
- Методы в категориях на чужих классах — тоже с префиксом: `gm_reversed`
- Конфликт имён классов = предупреждение рантайма и неопределённое поведение

Префикс	Откуда
`NS`	Foundation / AppKit (NeXTSTEP)
`GS`	расширения GNUstep
`UI`, `CA`, `CG`	UIKit, Core Animation, Core Graphics
`GM`	наша библиотека Geometry

Модуль = заголовок .h + реализация .m

04_modules/Geometry/GMCircle.h

```objc
#import <Foundation/Foundation.h>
#import "GMDefines.h"

@class GMPoint;                    // предварительное объявление вместо #import

GM_EXPORT
@interface GMCircle : NSObject
@property (nonatomic, strong, readonly) GMPoint *center;
@property (nonatomic, readonly) double radius;
- (instancetype)initWithCenter:(GMPoint *)center radius:(double)radius;
- (double)area;
- (BOOL)containsPoint:(GMPoint *)point;
@end
```

```
$ ls -R 04_modules
Geometry/  main.m  main_modules.m  Makefile  gnustep.mk
Geometry/: GMDefines.h  GMPoint.h  GMPoint.m
           GMCircle.h  GMCircle.m  Geometry.h  module.modulemap
```

- `.h` — интерфейс, `.m` — реализация
- `#import` не включает файл повторно
- `@class` — объявить класс без `#import`

Экспорт: что видно снаружи

04_modules/Geometry/GMPoint.h

```objc
#import <Foundation/Foundation.h>
#import "GMDefines.h"

NS_ASSUME_NONNULL_BEGIN

GM_EXPORT extern NSString *const GMGeometryVersion;   // публичная константа

GM_EXPORT
@interface GMPoint : NSObject <NSCopying>             // префикс GM вместо namespace
@property (nonatomic, readonly) double x;
@property (nonatomic, readonly) double y;
+ (instancetype)pointWithX:(double)x y:(double)y;
- (double)distanceTo:(GMPoint *)other;
@end

NS_ASSUME_NONNULL_END
```

04_modules/Geometry/GMDefines.h

```objc
// Экспорт символов из библиотеки: собираем с -fvisibility=hidden,
// а наружу открываем только то, что помечено GM_EXPORT.
#define GM_EXPORT __attribute__((visibility("default")))
```

Импорт и использование

04_modules/main.m

```objc
#import <Foundation/Foundation.h>
#import "Geometry.h"            // #import = #include, но не больше одного раза

int main(void) {
    @autoreleasepool {
        GMPoint  *a = [GMPoint pointWithX:0 y:0];
        GMPoint  *b = [GMPoint pointWithX:3 y:4];
        GMCircle *c = [[GMCircle alloc] initWithCenter:a radius:5];

        NSLog(@"Geometry v%@", GMGeometryVersion);
        NSLog(@"|%@ - %@| = %.1f", a, b, [a distanceTo:b]);
        NSLog(@"area=%.2f contains b: %d", [c area], [c containsPoint:b]);
        // b.x = 10;            // ошибка: свойство readonly снаружи модуля
    }
    return 0;
}
```

```
$ ./app_static
Geometry v1.0
|(0, 0) - (3, 4)| = 5.0
area=78.54 contains b: 1
```

Модули clang: @import

04_modules/Geometry/module.modulemap

```
module Geometry {
    umbrella header "Geometry.h"
    export *
}
```

04_modules/main_modules.m

```
@import Geometry;               // модуль: заголовки разбираются один раз и кэшируются
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        GMPoint *p = [GMPoint pointWithX:1 y:1];
        NSLog(@"@import works: %@, v%@", p, GMGeometryVersion);
    }
    return 0;
}
```

```
$ clang ... -fmodules -fmodules-cache-path=.modcache main_modules.m && ./app_modules
@import works: (1, 1), v1.0
```

Пакеты и системы сборки

Инструмент	Платформа	Роль
GNUstep Make	Linux, *BSD, Windows	система сборки: tool, library, framework, bundle
`apt install libgnustep-base-dev`	Debian / Ubuntu	готовые пакеты библиотек
Frameworks (`.framework`)	macOS / iOS	библиотека + заголовки + ресурсы в одной папке
CocoaPods, **Carthage**	macOS / iOS	менеджеры зависимостей
Swift Package Manager	macOS / iOS	поддерживает таргеты на Objective-C

04_modules/gnustep.mk

```
# по умолчанию GNUstep Make ищет файл GNUmakefile
MAKEFILE_NAME = gnustep.mk
GNUSTEP_MAKEFILES ?= $(shell gnustep-config --variable=GNUSTEP_MAKEFILES)
include $(GNUSTEP_MAKEFILES)/common.make

TOOL_NAME = geometry_demo
geometry_demo_OBJC_FILES = main.m Geometry/GMPoint.m Geometry/GMCircle.m
geometry_demo_OBJCFLAGS  = -fobjc-arc -IGeometry

include $(GNUSTEP_MAKEFILES)/tool.make
```

Декларация обобщённого класса

05_generics.m

```objc
@interface Stack<__covariant ObjectType> : NSObject     // параметр типа
@property (nonatomic, readonly) NSUInteger count;
- (void)push:(ObjectType)object;
- (nullable ObjectType)pop;
- (nullable ObjectType)peek;
@end
```

05_generics.m

```objc
@implementation Stack {
    NSMutableArray *_items;          // в реализации ObjectType недоступен -> id
}
- (instancetype)init {
    if ((self = [super init])) _items = [NSMutableArray array];
    return self;
}
- (NSUInteger)count { return _items.count; }
- (void)push:(id)object { [_items addObject:object]; }
- (id)pop {
    id last = _items.lastObject;
    if (last) [_items removeLastObject];
    return last;
}
- (id)peek { return _items.lastObject; }
@end
```

Инстанцирование и использование

05_generics.m

```
Stack<NSString *> *names = [Stack new];       // инстанцирование
[names push:@"Brad"];
[names push:@"Tom"];
NSString *top = [names pop];                  // без приведения типа
NSLog(@"pop=%@ count=%lu", top, (unsigned long)names.count);
// [names push:@42];  // warning: incompatible pointer types sending 'NSNumber *'

Stack<NSString *> *mutableNames = [Stack new];
Stack<NSObject *> *objects = mutableNames;    // __covariant: так можно
NSLog(@"covariant ok: %lu", (unsigned long)objects.count);

Registry<NSString *, NSNumber *> *ages = [Registry new];
[ages setValue:@39 forKey:@"Brad"];
NSLog(@"Brad: %@", [ages valueForKey:@"Brad"]);
```

```
$ ./05_generics
pop=Tom count=1
covariant ok: 0
Brad: 39
```

`[names push:@42]` → warning: incompatible pointer types sending `NSNumber *` to parameter of type `NSString *`

Generics стираются: проверки только при компиляции

05_generics.m

```
NSArray<NSString *> *strings = @[@"a", @"b"];
NSMutableArray *raw = [strings mutableCopy];   // generics стираются:
[raw addObject:@123];                          // рантайм ничего не проверяет
NSArray<NSString *> *again = raw;
for (id x in again) NSLog(@"%@ is %@", x, [x class]);
```

```
$ ./05_generics
a is GSTinyString
b is GSTinyString
123 is NSSmallInt
```

Lightweight generics нужны для подсказок компилятора и импорта в Swift. В рантайме `NSArray<NSString *>` — обычный `NSArray`, в нём может оказаться что угодно.

Коллекции Foundation

Неизменяемая	Изменяемая	Что это
`NSArray`	`NSMutableArray`	упорядоченный список, доступ по индексу
`NSSet`	`NSMutableSet`	множество уникальных объектов (по `isEqual:` / `hash`)
—	`NSCountedSet`	мультимножество: объект + число повторов
`NSOrderedSet`	`NSMutableOrderedSet`	уникальные элементы с сохранением порядка
`NSDictionary`	`NSMutableDictionary`	отображение ключ → значение, ключи копируются
`NSIndexSet`	`NSMutableIndexSet`	набор индексов
—	`NSHashTable`, `NSMapTable`	с поддержкой слабых ссылок

> В коллекциях — только объекты: числа → `NSNumber`, структуры → `NSValue`, «пусто» → `NSNull`.

06_collections.m

```m
int primes[5] = {2, 3, 5, 7, 11};           // C-массив: фиксированный размер,
size_t len = sizeof(primes) / sizeof(primes[0]);   // только примитивы,
printf("C array: len=%zu last=%d\n", len, primes[len - 1]);  // без проверки границ
```

```
$ ./06_collections
C array: len=5 last=11
```

NSArray

06_collections.m

```objc
NSArray<NSString *> *langs = @[@"Swift", @"C", @"Objective-C", @"Go"];
NSLog(@"count=%lu first=%@ [2]=%@", (unsigned long)langs.count, langs.firstObject, langs[2]);
NSLog(@"contains Go: %d, index of C: %lu", [langs containsObject:@"Go"],
      (unsigned long)[langs indexOfObject:@"C"]);
NSArray *sorted = [langs sortedArrayUsingSelector:@selector(compare:)];
NSArray *byLen  = [langs sortedArrayUsingComparator:^NSComparisonResult(NSString *a, NSString *b) {
    return [@(a.length) compare:@(b.length)];
}];
NSLog(@"sorted: %@", [sorted componentsJoinedByString:@", "]);
NSLog(@"by length: %@", [byLen componentsJoinedByString:@", "]);
NSLog(@"slice: %@", [[langs subarrayWithRange:NSMakeRange(1, 2)] componentsJoinedByString:@", "]);
```

```
$ ./06_collections
count=4 first=Swift [2]=Objective-C
contains Go: 1, index of C: 1
sorted: C, Go, Objective-C, Swift
by length: C, Go, Swift, Objective-C
slice: C, Objective-C
```

Множества

06_collections.m

```
NSSet *a = [NSSet setWithArray:@[@1, @2, @3, @3]];        // дубликаты исчезнут
NSMutableSet *u = [a mutableCopy];
[u unionSet:[NSSet setWithObjects:@3, @4, nil]];
NSMutableSet *inter = [a mutableCopy];
[inter intersectSet:[NSSet setWithObjects:@2, @3, @9, nil]];
NSLog(@"a=%lu items, union=%lu, intersect has 2: %d", (unsigned long)a.count,
      (unsigned long)u.count, [inter containsObject:@2]);

NSCountedSet *words = [[NSCountedSet alloc] initWithArray:
                       @[@"to", @"be", @"or", @"not", @"to", @"be"]];
NSLog(@"'to' x%lu, 'or' x%lu", (unsigned long)[words countForObject:@"to"],
      (unsigned long)[words countForObject:@"or"]);

NSOrderedSet *ordered = [NSOrderedSet orderedSetWithArray:@[@"b", @"a", @"b", @"c"]];
NSLog(@"ordered set: %@", [ordered.array componentsJoinedByString:@""]);
```

```
$ ./06_collections
a=3 items, union=4, intersect has 2: 1
'to' x2, 'or' x1
ordered set: bac
```

Словари

06_collections.m

```
NSMutableDictionary<NSString *, NSNumber *> *stock = [@{@"apple": @5} mutableCopy];
stock[@"pear"] = @2;                                    // добавить / заменить
[stock setObject:@7 forKey:@"apple"];
[stock removeObjectForKey:@"pear"];
stock[@"kiwi"] = @0;
NSLog(@"apple=%@ missing=%@", stock[@"apple"], stock[@"banana"]);   // nil
for (NSString *k in [stock keysSortedByValueUsingSelector:@selector(compare:)])
    NSLog(@"  %@: %@", k, stock[k]);
```

```
$ ./06_collections
apple=7 missing=(null)
  kiwi: 0
  apple: 7
```

Консоль: вывод

08_console.m

```objc
printf("printf -> stdout\n");                        // C stdio
fprintf(stderr, "fprintf -> stderr\n");
NSLog(@"NSLog -> stderr (+timestamp)");              // для отладки

NSFileHandle *out = [NSFileHandle fileHandleWithStandardOutput];
[out writeData:[@"NSFileHandle -> stdout\n" dataUsingEncoding:NSUTF8StringEncoding]];
```

```
$ ./08_console
printf -> stdout
fprintf -> stderr
NSLog -> stderr (+timestamp)
NSFileHandle -> stdout
```

- `stdout` — результат программы
- `stderr` — диагностика (`NSLog`)
- `NSFileHandle` — объектная обёртка над дескрипторами

Консоль: ввод

08_console.m

```
char buf[256];
long sum = 0;
int count = 0;
printf("Enter numbers (Ctrl+D to finish):\n");
while (fgets(buf, sizeof buf, stdin)) {              // строка за строкой
    NSString *line = [[NSString stringWithUTF8String:buf]
        stringByTrimmingCharactersInSet:[NSCharacterSet whitespaceAndNewlineCharacterSet]];
    NSScanner *sc = [NSScanner scannerWithString:line];
    NSInteger value;
    if ([sc scanInteger:&value] && sc.isAtEnd) {     // строгий разбор числа
        sum += value; count++;
    } else {
        fprintf(stderr, "skip: '%s'\n", line.UTF8String);
    }
}
printf("count=%d sum=%ld\n", count, sum);
```

```
$ printf "3\n5\nabc\n10\n" | ./08_console
Enter numbers (Ctrl+D to finish):
skip: 'abc'
count=3 sum=18
```

Файлы: запись

07_files.m

```
[fm createDirectoryAtPath:dir withIntermediateDirectories:YES attributes:nil error:&err];

NSString *text = @"first line\nsecond line\n";
BOOL ok = [text writeToFile:path atomically:YES
                   encoding:NSUTF8StringEncoding error:&err];   // весь файл сразу
NSLog(@"written: %d", ok);

NSFileHandle *fh = [NSFileHandle fileHandleForWritingAtPath:path];
[fh seekToEndOfFile];                                           // дозапись
[fh writeData:[@"third line\n" dataUsingEncoding:NSUTF8StringEncoding]];
[fh closeFile];
```

```
$ ./07_files
written: 1
```

Файлы: чтение

07_files.m

```
NSString *content = [NSString stringWithContentsOfFile:path
                                              encoding:NSUTF8StringEncoding error:&err];
NSArray *lines = [content componentsSeparatedByCharactersInSet:
                  [NSCharacterSet newlineCharacterSet]];
NSUInteger n = 0;
for (NSString *line in lines) {
    if (line.length == 0) continue;
    NSLog(@"%lu: %@", (unsigned long)++n, line);
}

NSData *bytes = [NSData dataWithContentsOfFile:path];           // двоичный доступ
NSLog(@"size=%lu bytes", (unsigned long)bytes.length);
```

```
$ ./07_files
1: first line
2: second line
3: third line
size=34 bytes
```

Итоги второй части

- **NSException** — для логических ошибок в коде; **NSError** — для штатных сбоев (нет файла, неверный ввод)
- `NSAssert`, `NSParameterAssert`, `assert` — отключаются макросами в релизе
- Нет namespace → префиксы; модуль = `.h` + `.m`; `static`, расширения классов
- Экспорт через заголовки, `@import` и module map, GNUstep Make
- Generics — только для компилятора, в рантайме стираются
- Коллекции: пары immutable/mutable, только объекты
- Ввод-вывод: C stdio + `NSFileHandle`, `NSFileManager`, JSON

Источники

1. Tutorialspoint. Objective-C Tutorial
   https://www.tutorialspoint.com/objective_c/index.htm
2. Feifan Zhou. Learn Objective-C in 24 Days (Binpress)
   https://www.binpress.com/learn-objective-c-24-days/
3. Apple. Programming with Objective-C
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ProgrammingWithObjectiveC/Introduction/Introduction.html
4. Apple. Error Handling Programming Guide
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/ErrorHandlingCocoa/ErrorHandling/ErrorHandling.html
5. Apple. Exception Programming Topics
   https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/Exceptions/Exceptions.html
6. Clang. Modules
   https://clang.llvm.org/docs/Modules.html
7. GNUstep Base (Foundation)
   https://github.com/gnustep/libs-base
8. GNUstep Make
   https://github.com/gnustep/tools-make